

极化码

peterrolls@qq.com

2026 年 5 月 30 日

极化码将编码的过程模拟为信道传输，其思想是“好信道传信息，差信道传已知比特”。

1 编码

编码的步骤是：

1. 确定可靠子信道：通过巴氏（Bhattacharyya）参数或密度进化算法选择 K 个最可靠的信道位置；
2. 构造输入向量 u ：在可靠的位置放置信息比特 $s_1, s_2, \dots, s_n$ ，冻结位置放置约定好的已知值；
3. 使用矩阵乘法或者编码结构生成码字。

1.1 信道合并

N 个独立的二进制离散信道 W 使用递归的方式生成 $W_N : x^N \rightarrow y^N$ ，其中 $N = 2^n, n \geq 0$ 。通过递归的方式合并信道，从第0级($n = 0$)开始。

1.1.1 第0级($n = 0$)

$n = 0$ ，一个独立的二进制离散信道 W ，定义 $W_1 = W$ 。表示一个二进制离散无记忆信道。

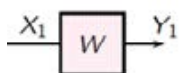


图 1: 第0级信道合并结构图

1.1.2 第1级($n = 1$)

$n = 1$ ，合并两个独立的二进制离散信道 W_1 ，得到 $W_2 : x^2 \rightarrow y^2$ ， W_2 的转移概率为：

$$W_2(y_1, y_2 | u_1, u_2) = W(y_1 | u_1 \oplus u_2) W(y_2 | u_2)$$

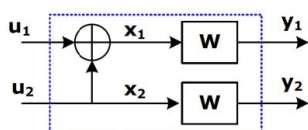


图 2: 第1级信道合并结构图

1.1.3 第2级($n = 2$)

将两个 $n = 1$ 的 $W_2 : x^2 \rightarrow y^2$ 信道复合得到 $W_4 : x^4 \rightarrow y^4$, W_4 的转移概率为:

$$\begin{aligned} W_4(y_1^4|u_1^4) &= W_2(y_1^2|u_1^2 \oplus u_2^2, u_3^2 \oplus u_4^2)W_2(y_3^2|u_2, u_4) \\ &= W(y_1|u_1 \oplus u_2 \oplus u_3 \oplus u_4)W(y_2|u_3 \oplus u_4)W(y_3|u_2 \oplus u_4)W(y_4|u_4) \\ &= W^4(y_1^4|u_1^4 G_4) \end{aligned}$$

这里:

$$G_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

将结构图的交叉扭转, 可以得到其等效图:

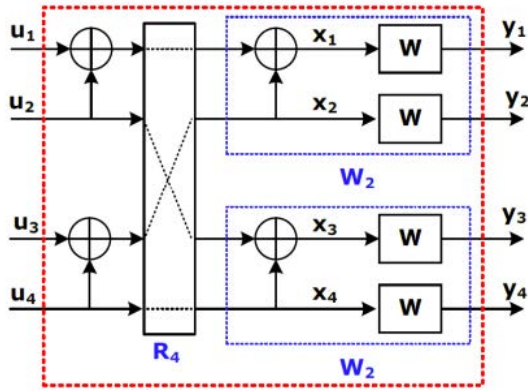


图 3: 第2级信道合并结构图

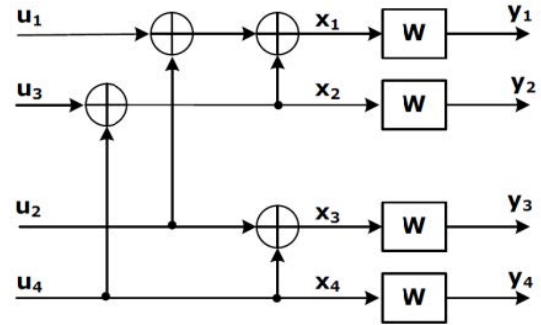


图 4: 第2级信道合并等效结构图

1.1.4 第3级($n = 3$)

$n = 3$ 时有 $W_8 : x^8 \rightarrow y^8$, 由两个 $n = 2$ 的 $W_4 : x^4 \rightarrow y^4$ 信道复合得到。

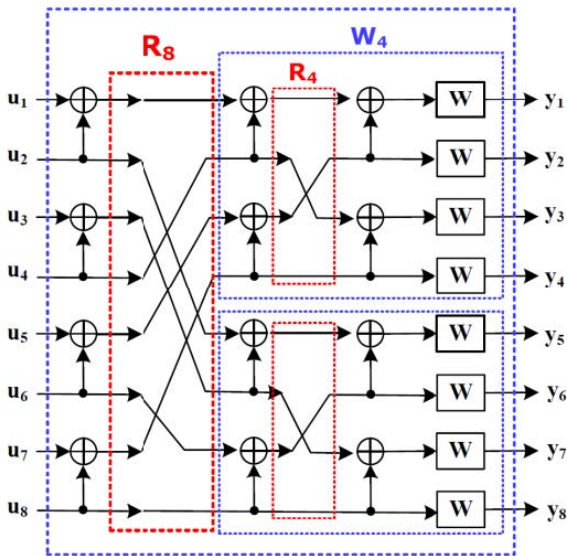


图 5: 第3级信道合并结构图

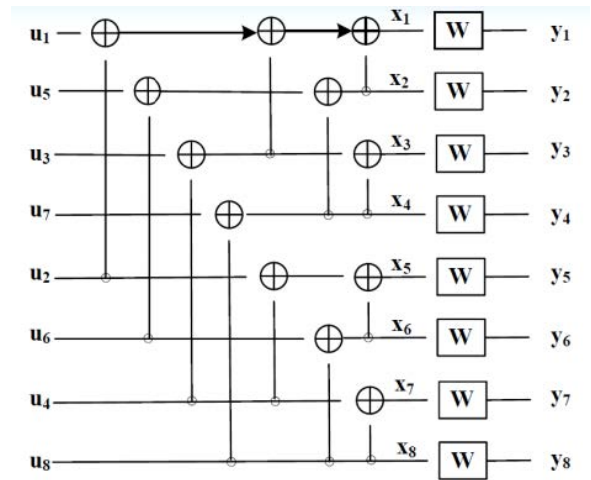


图 6: 第3级信道合并等效结构图

1.2 编码矩阵构造

编码矩阵由克罗内克(Kronecker)积构造:

$$\begin{aligned} G_N &= B_N F^{\otimes n} \\ B_N &= R_N (I_2 \otimes B_{N/2}) \\ F &= \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \end{aligned}$$

$F^{\otimes n}$ 是 F 的 n 次克罗内克积。克罗内克(Kronecker)积是这样的运算, 设 $A = (a_{ij})_{N \times N}$:

$$A \otimes B = \begin{bmatrix} a_{11}B & a_{12}B & \cdots & a_{1N}B \\ a_{21}B & a_{22}B & \cdots & a_{2N}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{N1}B & a_{N2}B & \cdots & a_{NN}B \end{bmatrix}$$

R_N 是一个置换矩阵, 实现奇数位放在前面, 偶数位放在后面:

$$\begin{aligned} \begin{bmatrix} 1 & 3 & 2 & 4 \end{bmatrix} &= \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} R_4 \\ \begin{bmatrix} 1 & 3 & 5 & 7 & 2 & 4 & 6 & 8 \end{bmatrix} &= \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \end{bmatrix} R_8 \end{aligned}$$

这里:

$$R_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} R_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} R_8 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

实际上 G_N 也可以用迭代的方式构建:

$$G_2 = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} G_{2Z} = \underbrace{\begin{bmatrix} 1 & \mathbf{0} & 0 & \mathbf{0} \\ 1 & \mathbf{0} & 1 & \mathbf{0} \end{bmatrix}}_{\text{每个元素后添加0}} G_{2R} = \underbrace{\begin{bmatrix} 1 & \mathbf{1} & 0 & \mathbf{0} \\ 1 & \mathbf{1} & 1 & \mathbf{1} \end{bmatrix}}_{\text{每个元素重复}}$$

然后将 G_{2Z} 和 G_{2R} 组合, 得到 G_4 :

$$G_4 = \begin{bmatrix} G_{2Z} \\ G_{2R} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

同理也可以构建 G_8 :

$$G_{4Z} = \begin{bmatrix} 1 & \mathbf{0} & 0 & \mathbf{0} & 0 & \mathbf{0} & 0 & \mathbf{0} \\ 1 & \mathbf{0} & 0 & \mathbf{0} & 1 & \mathbf{0} & 0 & \mathbf{0} \\ 1 & \mathbf{0} & 1 & \mathbf{0} & 0 & \mathbf{0} & 0 & \mathbf{0} \\ 1 & \mathbf{0} & 1 & \mathbf{0} & 1 & \mathbf{0} & 1 & \mathbf{0} \end{bmatrix} G_{4R} = \begin{bmatrix} 1 & \mathbf{1} & 0 & \mathbf{0} & 0 & \mathbf{0} & 0 & \mathbf{0} \\ 1 & \mathbf{1} & 0 & \mathbf{0} & 1 & \mathbf{1} & 0 & \mathbf{0} \\ 1 & \mathbf{1} & 1 & \mathbf{1} & 0 & \mathbf{0} & 0 & \mathbf{0} \\ 1 & \mathbf{1} & 1 & \mathbf{1} & 1 & \mathbf{1} & 1 & \mathbf{1} \end{bmatrix}$$

$$G_8 = \begin{bmatrix} G_{4Z} \\ G_{4R} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

极化码的生成矩阵自身就是自身的逆矩阵，即 $G_N = G_N^{-1}$ 。

1.3 编码

可以直接通过矩阵乘得到编码结果：

$$\vec{v} = \vec{u} \cdot G$$

其复杂度是 $O(N^2)$ 。

按照编码图中的结构会更快，其复杂度是 $O(N \log_2 N)$

1.4 极化信道可靠性估计

对于BEC（二进制擦除信道），可以计算巴氏参数 $Z(W_N^{(i)})$ ；对于其他信道，如BSC（二进制对称信道）或AWGNC（加性高斯白噪声信道），不能得到精确的巴氏参数，可采用密度进化或高斯近似估计法估计信道的可靠性。

对于BEC来说，选择巴氏参数最小的 K 个子信道放置消息比特。

给定一个二进制离散无记忆信道(BDMC) $W: X \rightarrow Y$ ， X 和 Y 分别为输入和输出。令 $P(Y|X)$ 为信道转移概率，其中 $X \in \{0, 1\}$ 。

对于信道 W ，信道容量 $I(W)$ 为：

$$I(W) = \sum_{y \in Y} \sum_{x \in X} \frac{1}{2} P(y|x) \log \frac{P(y|x)}{\frac{1}{2} P(y|0) + \frac{1}{2} P(y|1)}$$

其巴氏(Bhattacharyya)参数：

$$Z(W) = \sum_{y \in Y} \sqrt{P(y|0)P(y|1)}$$

信道容量是等概信源通过信道 W 可靠传输的最高速率；

巴氏参数是信道使用一次最大似然判决错误概率的上限。

$I(W)$ 和 $Z(W)$ 取值范围都为 $[0, 1]$ ，当 $I(W) \approx 1$ 时， $Z(W) \approx 0$ ，当 $Z(W) \approx 1$ 时， $I(W) \approx 0$ 。

使用递归运算计算BEC的Bhattacharyya参数：

$$\begin{aligned} Z(W_N^{2k-1}) &= 2Z(W_{\frac{N}{2}}^k) - [Z(W_{\frac{N}{2}}^k)]^2 \\ Z(W_N^{2k}) &= [Z(W_{\frac{N}{2}}^k)]^2 \end{aligned}$$

初始值为：

$$\begin{aligned} Z(W_1^1) &= \sum_{y \in Y} \sqrt{P(y|0)P(y|1)} \\ &= \sqrt{0 \cdot (1-p)} + \sqrt{p \cdot p} + \sqrt{(1-p) \cdot 0} \\ &= p \end{aligned}$$

2 译码

采用逐次消去译码算法进行译码，通过符号的似然比进行二进制判决。

2.1 基本译码单元

极化码的编码方向和解码方向是相反的，也因此可以利用符号间的概率联系进行似然比的计算：其

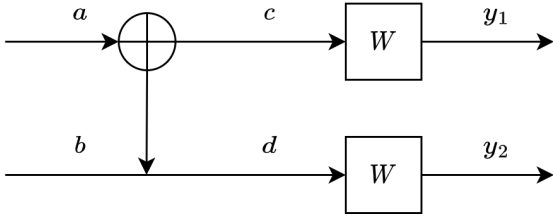


图 7: 极化码编码器的最小单元

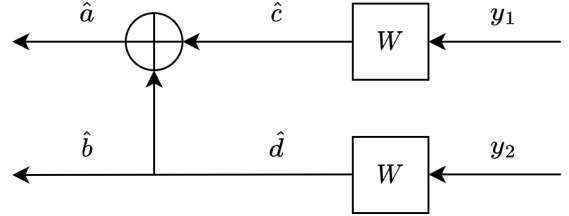
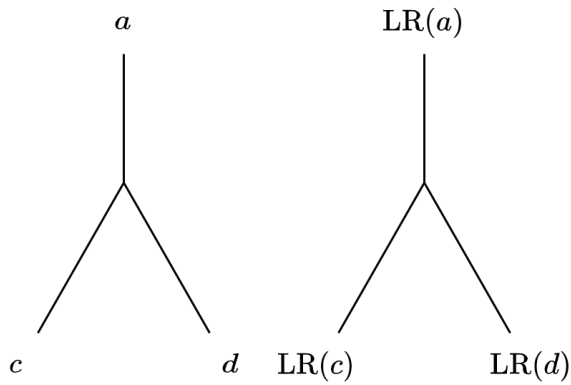


图 8: 极化码译码器的最小单元

中， $\hat{c}$, $\hat{d}$ 是待估计的编码符号； $\hat{a}$, $\hat{b}$ 是原始发送信息比特的估计值； W 是信道输入输出之间的转移概率矩阵。

首先进行 $\hat{a}$ 的译码， $\hat{a} = \hat{c} + \hat{d}$ ：得到 $\hat{a}$ 的似然比：



$\hat{c}$	$\hat{d}$	$\hat{a}$
0	0	0
0	1	1
1	0	1
1	1	0

表 1: $\hat{a}$ 的状态表

图 9: 极化码译码过程中的似然比计算单元

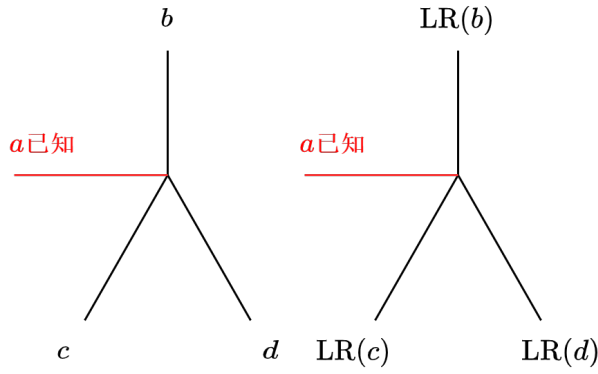
$$\begin{aligned}
 \text{LR}(\hat{a}) &= \frac{P(\hat{a} = 0)}{P(\hat{a} = 1)} \\
 &= \frac{P(\hat{c} = 0)P(\hat{d} = 0) + P(\hat{c} = 1)P(\hat{d} = 1)}{P(\hat{c} = 0)P(\hat{d} = 1) + P(\hat{c} = 1)P(\hat{d} = 0)} \\
 &= \frac{1 + \text{LR}(\hat{c})\text{LR}(\hat{d})}{\text{LR}(\hat{c}) + \text{LR}(\hat{d})}
 \end{aligned}$$

在已经译出 $\hat{a}$ （或已知 $\hat{a}$ 的似然比）的情况下，现在对 $\hat{b}$ 进行译码：

$$\begin{cases} \hat{b} = \hat{d} \\ \hat{a} = \hat{c} + \hat{d} = \hat{c} + \hat{a} \end{cases}$$

从而 $\hat{b} = \hat{a} + \hat{c}$ ：分别考虑 $\hat{a} = 0$ 和 $\hat{a} = 1$ 两种情况，可以计算出：

$$\text{LR}(\hat{b}) = \begin{cases} \text{LR}(\hat{c})\text{LR}(\hat{d}) & \text{当 } \hat{a} = 0 \\ \frac{\text{LR}(\hat{d})}{\text{LR}(\hat{c})} & \text{当 } \hat{a} = 1 \end{cases} = [\text{LR}(\hat{c})]^{1-2\hat{a}} \text{LR}(\hat{d})$$



$\hat{a}$	$\hat{c}$	$\hat{d}$	$\hat{b}$
0	0	0	0
1	0	1	1
1	1	0	0
0	1	1	1

表 2: $\hat{b}$ 的状态表

图 10: 极化码译码过程中的似然比计算单元

2.2 二进制删除信道(BEC)

二进制删除信道的转移概率如图所示： 其中的-1代表被擦除的符号，可以理解成非法符号，其输出

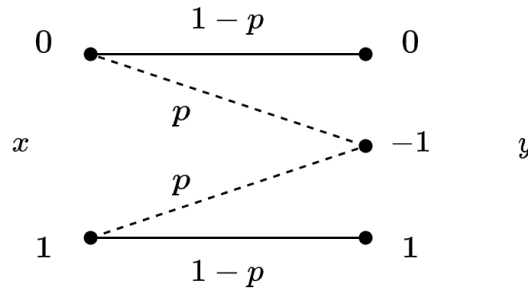


图 11: 二进制删除信道的转移概率

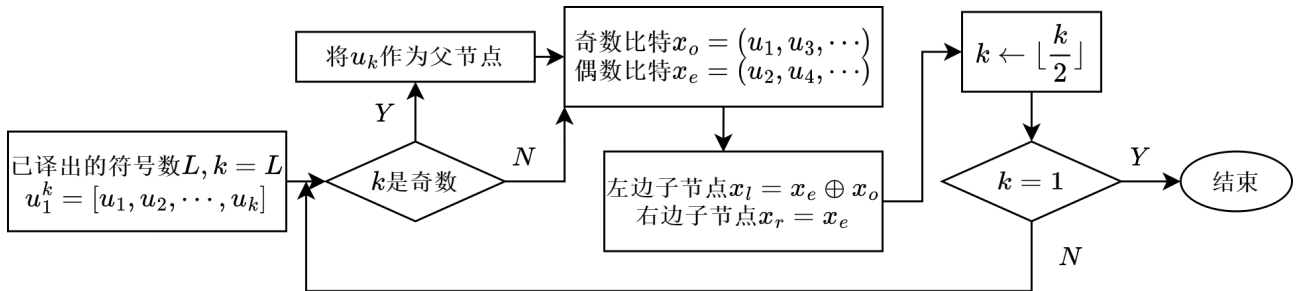
似然比为：

$$LR_k = \frac{P(x=0|y=k)}{P(x=1|y=k)} = \frac{P(y=k|x=0)P(x=0)}{P(y=k|x=1)P(x=1)}$$

当计算得到无穷大时，可设置为一个较大的值，比如100，用于树图中连续计算。

2.3 译码步骤

节点分布情况通过如下流程图进行计算：



也可以基于生成矩阵计算已译码符号在各节点上的分布：

设已译码的符号为 $u_1^k = [u_1, u_2, \dots, u_k]$, $k < N$, 将 k 按二进制展开: $k = \underbrace{2^{i_m}}_{k_m} + \underbrace{2^{i_{m-1}}}_{k_{m-1}} + \dots + \underbrace{2^{i_1}}_{k_1}$,

接收符号按二进制位数对应每个 k_m :

$$\underbrace{u_1 u_2 \cdots}_{k_m} \underbrace{\cdots}_{k_{m-1}} \cdots \underbrace{\cdots}_{k_1} u_k$$

顺次取出长度为 k_i 的向量 $\vec{v}_i$, $i = 1, 2, \dots, m$, 从而有 $\vec{b}_i = \vec{v}_i \cdot G_{k_i}$

基于树图的连续消除译码算法包括以下两个步骤:

1. 给出已译码信息符号在树图各节点的分布情况 (画出树图);
2. 对当前信息符号进行译码 (使用基本译码单元计算似然比)。

举例说明: $N = 16$, $u_1^7 = [u_1 u_2 u_3 u_4 u_5 u_6 u_7]$, 现对 u_8 进行译码, 计算已译出比特的节点分布。

因为 $7 = (111)_2 = 2^2 + 2^1 + 2^0$, $\underbrace{u_1 u_2 u_3 u_4}_4 \underbrace{u_5 u_6}_2 \underbrace{u_7}_1$, 有:

$$\vec{v}_4 = [u_1 u_2 u_3 u_4] \quad \vec{v}_2 = [u_5 u_6] \quad \vec{v}_1 = [u_7]$$

$$\vec{b}_4 = \vec{v}_4 G_4 = \begin{bmatrix} u_1 + u_2 + u_3 + u_4 & u_3 + u_4 & u_2 + u_4 & u_4 \end{bmatrix}$$

$$\vec{b}_2 = \vec{v}_2 G_2 = \begin{bmatrix} u_5 + u_6 & u_6 \end{bmatrix}$$

$$\vec{b}_1 = \vec{v}_1 G_1 = \begin{bmatrix} u_7 \end{bmatrix}$$

绘制出的节点分布情况如下图所示:

